Project report Group 12
Risk-aware trajectory planning under multi-modal environmental uncertainty in autonomous driving

Albin Cederberg, Alexander Gustafsson, Aravind Jawahar,
Hannes Lundberg, Jared Tittus, Yibo Zhou

***Abstract*—Autonomous driving in urban environments requires motion planning methods that can operate safely under uncertain and multi-modal behavior of surrounding vehicles, particularly in interaction-critical scenarios such as intersections. This paper addresses risk-aware trajectory planning for an ego vehicle (EV) when the future behavior of an obstacle vehicle is uncertain. We investigate two model predictive control (MPC) frameworks that explicitly address multi-modal uncertainty in interactive driving scenarios. First, a Contingency MPC (C-MPC) framework is considered, which augments a nominal MPC policy with contingency actions to ensure safety under unexpected deviations in obstacle behavior. Building upon this formulation, a Single-Branch Bayesian MPC (B-MPC) is introduced, which further incorporates probabilistic predictions of obstacle motion into a chance-constrained MPC framework, enabling systematic risk-aware planning with controlled conservatism. Both controllers are evaluated under identical settings in a synthetic intersection simulation and in a data-driven environment constructed from the inD dataset, demonstrating applicability to realistic traffic scenarios. Simulation results show that B-MPC can safely manage many uncertain interactions, while the inclusion of contingency planning significantly enhances robustness. In scenarios involving unexpected obstacle acceleration, the proposed C-MPC reduces the collision rate from 24% to 4% without affecting nominal performance. The results indicate that combining probabilistic prediction with chance-constrained and contingency-aware MPC provides an effective and practical solution for autonomous driving under multi-modal environmental uncertainty.**

***Index Terms*—Autonomous driving, Model predictive control, Risk-aware planning, Multi-modal uncertainty, Contingency planning.**

I. Introduction

Autonomous driving faces significant challenges when operating in fast-changing and highly uncertain environments, especially in scenarios involving multi-modal behaviour of other vehicles (OVs) (e.g. intersection, overtaking). Ensuring safety in such settings demands risk-aware planning frameworks that strike a balance between safety guarantees, conservativeness, and computational tractability. Existing risk-constrained methods face limitations:

- Nominal formulations may be efficient but can lose safety under uncertainty.
- Robust formulations guarantee feasibility but are often overly conservative, leading to suboptimal or infeasible plans.
- Contingency and sample-based methods reduce conservatism but increase computational complexity, limiting real-time applicability

A. Purpose

The purpose of this project is to implement and evaluate state of the art control strategies using simulations and real world data. This will give insight into pros and cons of different control strategies and risk metrics. Simulations will hopefully highlight conceptual differences while implementations using real world data might reveal practical challenges.

B. Problem/Task

The problem to be tackled in this project is to design a safe way to operate an autonomous vehicle surrounded by OV. In situations such as intersections, lane merges, and overtaking, OV may exhibit multiple possible future actions, e.g., yielding or continuing forward, which creates a multi-modal uncertainty in trajectory prediction. Incorrect assumptions about which motion will occur can lead to unsafe or overly conservative behaviour.

Thus, the problem of this project can be described as: How an autonomous vehicle can plan safe, feasible trajectories when the future behaviour of OV is uncertain. The overall problem is divided into two major components: a theoretical component and a simulation-based evaluation component.

Theoretical Component.:

- To build a prediction model that captures multiple possible future trajectories of the OV, enabling the controller to reason about multi-modality and uncertainty.

- Based on this prediction model, we construct two MPC frameworks:
  1) **Contingency MPC (C–MPC)**, which generates a Contingency plan in case unlikely scenarios occurs.
  2) **Single branch baysian MPC (B–MPC)**, which branches the optimization into multiple prediction-consistent subproblems to explicitly handle trajectory uncertainty.
- For safety constraint a **Gaussian Chance Constraint (GCC)** is to be explored, which incorporates probabilistic safety margins to achieve constraint satisfaction with a predefined probability.

Simulation Component: To evaluate and compare the proposed control strategies, two complementary simulation environments are developed:

1) **Synthetic simulation environment**: A controlled setup designed to model predefined vehicle encounter scenarios at intersections. This environment allows systematic testing of specific traffic configurations and edge cases.
2) **inD-based simulation environment**: A data-driven environment constructed using the inD dataset, enabling the replication of real-world road geometries, traffic flows, and complex multi-vehicle interactions.

In both simulation environments, the EV is controlled by the MPC algorithms, while the OVs follow either synthetically generated or dataset-derived trajectories. Both the B-MPC and the C-MPC are deployed under identical conditions to ensure a fair comparison.

The simulations assess whether the EV can maintain safe distances and successfully avoid collisions under varying traffic conditions. Performance is evaluated and compared across the two MPC formulations, with particular emphasis on safety, robustness to prediction errors, and control efficiency.

C. Limitations

The primary limitation is that the project will rely exclusively on existing methods, frameworks, and open-source libraries. No new algorithms or predictive models will be developed from scratch. Instead, the emphasis is on implementing, and evaluating already published techniques such as C-MPC, B-MPC, and chance-constrained or conditional value at risk (CVaR) formulations of MPC.

Also, this project will be based on simulations and no real-world implementation will be done. However, real world data from the InD dataset will be used in these simulations.

The computational demands of the different MPCs won't be taken into consideration in the evaluation of performance.

The project will exclusively focus on longitudinal control, meaning that the EVs velocity will be controlled. The heading of the vehicles will therefore not change during simulations. The scenario chosen is 2 cars approaching an intersection, perpendicular to each other.

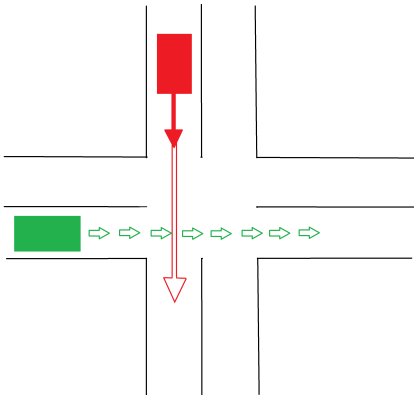

Fig. 1: Simulated scenario, The green car is EV and the red car is OV. The green arrows shows our desired path and hollow red arrow indicates the uncertain path of the OV.

II. Method

To increase efficiency throughout the project, the group will be split into three smaller groups where the tasks will be:

- Predict the future actions of the OV.
- Design a B-MPC.
- Design a C-MPC.

A. Predicting future actions of OV

To support the planning approaches for the B-MPC and C-MPC, a solution will be designed to estimate the future system states of the OV and the probabilities for certain actions. The probability of the actions can then be used by the BMPC and CMPC to control the EV.

1) Binary prediction: A simple version where the OV has 2 actions(stop/go) at each time-step is visualized in Fig. 6.

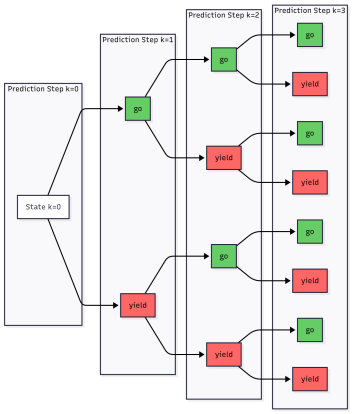

Fig. 2: Example structure of trajectory branches. At each time step a prediction is made whether the OV will yield or go.

A simple binary predictor of this type was created using a threshold for the acceleration $a_{lim} = -0.5m/s^2$ to determine the future action of either *go* or *yield*. In both cases the coordinated turn (CT) model is used but with a slight variation when the OV is predicted to yield.

$$\mathbf{x}_k = \begin{cases} x_k &= x_{k-1} + v_{k-1}\Delta t \cdot \cos(\varphi_{k-1}) \\ y_k &= y_{k-1} + v_{k-1}\Delta t \cdot \sin(\varphi_{k-1}) \\ v_k &= v_{k-1} \\ \varphi_k &= \varphi_{k-1} + \omega_{k-1}\Delta t \\ \omega_k &= \omega_{k-1} \end{cases}$$

If the current acceleration $a_k > a_{lim}$, the initial conditions for the model is simply set at as the current state $\mathbf{x}_k$. In the other case where $a_k < a_{lim}$, the OV is predicted to stop by the stop line. This is done by calculating the required constant deceleration a_{ret} to stop at the line. The calculation can be seen in equation 1, where d_{stop} is the distance between the OV and the line. The time for prediction horizon is T_{pred}. When the future trajectory is calculated the velocity is not set as constant like in the CT model and is instead calculated as $v_k = v_{k-1} + a_{ret}\Delta t$, where the acceleration is negative.

$$\begin{aligned} &\frac{a_{ret}T_{pred}^2}{2} + v_k T_{pred} = d_{stop} \\ \implies\ &a_{ret} = 2\left(\frac{-v_k}{T_{pred}} + \frac{d_{stop}}{T_{pred}^2}\right) \end{aligned} \tag{1}$$

2) Trajectron++: The binary predictor is simple and computationally efficient; however, it cannot quantify the uncertainty of the other vehicles beyond a binary choice. This limitation becomes a bigger problem with longer horizons and in situations where multiple possible future behaviors coexist. Since both BMPC and CMPC rely on probabilistic, multi-modal predictions, a more versatile prediction model is needed.

Ideally, the requirements that could satisfy the chosen controllers can be summed up as follows:

- Multi-modality: Ability to produce behaviorally distinct trajectory hypotheses
- Probabilistic Outputs: Ability to provide the likelihood of each predicted trajectory.
- Real world relevance: Must be trained on real-world training data to capture human driving behavior, not just arbitrary kinematics.
- Practical feasibility: Must be feasible to implement within the project scope, leveraging an existing framework.

Several open source trajectory prediction models were reviewed to determine how they represent uncertainty and interactions. Social-LSTM and Social-GAN are early sequence-based models that can generate multiple future samples but do not provide calibrated probabilities for distinct trajectories, which limits their applicability in risk-aware planning and chance-constrained MPC [1, 2]. Models like PECNET that are goal-based methods are capable of predicting diverse trajectories over a long horizon; however, it needs processing to produce probabilistic mode representations suitable for MPC formulations [3].

A number of prediction models developed specifically for vehicle motion address multi-modality more explicitly. Models such as CoverNet, MultiPath and MultiPath++ are based on trajectory set and anchor-based methods that are capable of predicting a discrete set of future trajectories and corresponding probabilities, making them well-suited for branching planners [4, 5, 6]. However, some of them are not open source, and they are originally trained on carefully constructed anchor trajectories for large datasets, which could be challenging to adopt into other datasets or test environments. Though map-aware graph-based models such as VectorNet, LaneGCN, and DenseTNT perform well in autonomous driving benchmarks, they do not provide probabilities of future trajectories in a directly usable form [7, 8, 9].

Trajectron++ was therefore chosen as it meets the project requirements with minimal additional development effort. It represents future uncertainty using a discrete latent variable model, yielding multiple distinct trajectory hypotheses together with explicit probability assignments.[10]. It also has the capability of modeling inter-agent interactions, this feature will however not be used in this project. The modes of Trajectron++ can be used as branches, and weighted trajectory statistics can be used as chance constraints. It is also open source and provides pre-trained models based on different standardized large datasets. A drastic simplification of the Trajectron++ can be seen below 3.

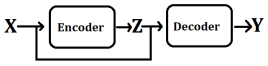

Fig. 3: Drastic simplification of Trajectron++

The model generates 25 different driving modes Z, that are modeled using an encoder, with the input being the states of all present OV. Since only one OV is present in this project X only contains the past states of one OV and the map data. The model can handle multiple OV, meaning that X can include the history of multiple road users. The states include the same states as in the CT model from the current time k to $k-h$. After the Trajectron++ has generated the modes, it passes them together with the states to the decoder, where the trajectories Y are generated. The trajectories contain sequences of 2D-positional coordinates for every OV. The output can be explained statistically as seen below.

$$z \sim p(z \mid X), \qquad y \sim p(y \mid X, z). \tag{2}$$

Where z is one of the modes and y is one of the trajectories that are generated. This gives us 25 different trajectories per OV and their corresponding probability of the mode that generated it. Trajectories with a probabilities that are less than 0.1% are ignored by the controllers. The most probable trajectories at a certain time step can be seen in figure 4. The lines seen are the means of the different probability distributions.

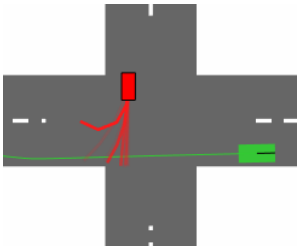

Fig. 4: The most probable trajectories of an OV at a certain time step, with a prediction horizon of 2.5 seconds

The model is pretrained on the nuScenes data set. The data set is based of real world data collected by a driving car equipped with cameras, radar and lidars. To be able to use the model on our simple Matlab simulation, there was a need to manipulate the data from the Matlab simulation to make it as similar as possible to the input the model has received while being trained on the nuScenes data set. This involved creating smaller patches of the map that are centered

around the OV. These smaller map patches also need to be rotated to match the OV-heading. Even if the training data is gathered from a sensor system, the data is fused and the state estimation is what is used to train the model. This means that there is no need to simulate the sensors in the Matlab simulation and the OV states can be used as they are.

B. Generic MPC Framework

To enable a fair and consistent comparison between the considered control methods, a generic model predictive control (MPC) framework is defined in this section. This framework specifies all components that are shared across controllers, including the ego vehicle dynamics, reference trajectory generation, prediction horizon, and core MPC formulation. By fixing these elements, any observed differences in closed-loop behavior can be attributed solely to the controller-specific objectives and constraints introduced in later sections.

1) Dynamics model of the Ego Vehicle: The dynamics of the vehicle can be described in two formats, a non-linear model that captures the exact kinematic evolution of the system, and a linearized representation that enables efficient prediction and optimization within a MPC framework. Since the EV operates in a planar environment and the focus of this work is on motion planning and collision avoidance rather than low-level actuation, a non-linear kinematic modeling approach is adopted.

The non-linear model is used to accurately represent the vehicle motion, while its linearization is employed within the MPC optimization to ensure computational tractability of the resulting problem. This formulation allows the controller to retain fidelity to the true vehicle behavior while maintaining real-time feasibility. Non-linear Extended Coordinated Turn Model

For the EV, as shown in Fig. 5, we adopt an extended CT motion model that augments the standard CT dynamics with longitudinal acceleration as a control input. This allows the EV to actively regulate its speed, while the yaw-rate input remains consistent with the conventional CT formulation. Defining p_t^x and p_t^y as the vehicle's position, v_t as the longitudinal speed, ϕ_t as the heading angle, a_t as the longitudinal acceleration input, and ω_t as the yaw-rate input. The resulting discrete-time model with sampling time T is given by

$$\begin{aligned}
\mathbf{x}_{t+1} &= \begin{bmatrix} p_t^x + \left(Tv_t + \frac{T^2}{2}a_t\right)\cos(\phi_t + T\omega_t) \\ p_t^y + \left(Tv_t + \frac{T^2}{2}a_t\right)\sin(\phi_t + T\omega_t) \\ v_t + Ta_t \\ \phi_t + T\omega_t \end{bmatrix} \\
\mathbf{x}_t &= \begin{bmatrix} p_t^x \\ p_t^y \\ v_t \\ \phi_t \end{bmatrix} \\
\mathbf{u}_t &= \begin{bmatrix} a_t \\ \omega_t \end{bmatrix},
\end{aligned} \tag{3}$$

Compared to the standard CT model and other linear motion model, this extension provides additional

control authority over the EV's longitudinal motion, making it better suited for collision avoidance and speed regulation within the MPC framework.

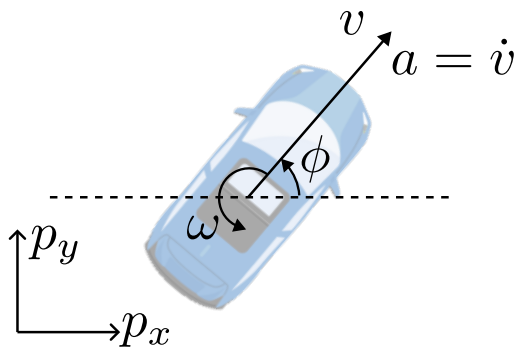

Fig. 5: Extended Coordinated Turn dynamics model for the EV.

2) Linearization of the Nonlinear Model: The non-linear EV dynamics in (3) are linearized at each MPC iteration around the previously predicted state and input $(\bar{\mathbf{x}}_t, \bar{\mathbf{x}}_t)$. The resulting Jacobian-based linear model is

$$\mathbf{x}_{t+1} = A_t \mathbf{x}_t + B_t \mathbf{u}_t + \delta_t \tag{4}$$

where the system matrices are obtained as

$$\begin{aligned} A_t &= \begin{bmatrix} 1 & 0 & T\cos\theta_t & -d_t \sin\theta_t \\ 0 & 1 & T\sin\theta_t & d_t \cos\theta_t \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\ B_t &= \begin{bmatrix} \frac{T^2}{2}\cos\theta_t & -T\, ds_t \sin\theta_k \\ \frac{T^2}{2}\sin\theta_t & T\, ds_t \cos\theta_t \\ T & 0 \\ 0 & T \end{bmatrix} \end{aligned} \tag{5}$$

with

$$ds_t = T v_t + \frac{T^2}{2} a_t, \qquad \theta_t = \phi_t + T\omega_t \tag{6}$$

To compensate for the mismatch between the non-linear model $f(\mathbf{x}_t, \mathbf{u}_t)$ and its linear approximation $A_t\mathbf{x}_t + B_t\mathbf{u}_t$, an affine correction term δ_t is included. In principle, the linearization is performed around the current predicted state and input $(\bar{\mathbf{x}}_t, \bar{\mathbf{u}}_t)$. Therefore, the affine term is computed as

$$\delta_t = f(\bar{\mathbf{x}}_t, \bar{\mathbf{u}}_t) - A_t\bar{\mathbf{x}}_t - B_t\bar{\mathbf{u}}_t.$$

This correction ensures that the linear model matches the non-linear dynamics exactly at the linearization point and remains locally accurate throughout the prediction horizon.

3) Vehicle Model and Parameters: The EV itself is based on the most sold car in Europe 2024 Dacia Sandrero [11], a maximum de-acceleration and acceleration was calculated on data from fastestlaps.com [12]. The de-acceleration was calculated by taking the average de-acceleration needed to manage the $150-0[kph]$ in $86[m]$ which results in a maximum de-acceleration of $-10.0937[m/s^2]$. For the acceleration a non-linear model wasn't able to be used so the mean acceleration from $50-80[kph]$ in $4.4[s]$ was calculated to be $1.8939[m/s^2]$.

C. Reference Trajectory Generation

A constant reference speed V_{ref} and a constant reference heading ϕ_{ref} are specified to define the desired motion of the EV. Using these quantities, the reference trajectory X_{ref} over the prediction horizon is generated as a straight-line motion. The reference position evolves according to

$$\begin{aligned} p_{x,\text{ref},k} &= p_{x,t} + kTV_{\text{ref}} \cos\phi_{\text{ref}} \\ p_{y,\text{ref},k} &= p_{y,t} + kTV_{\text{ref}} \sin\phi_{\text{ref}} \end{aligned} \tag{7}$$

where (x_t, y_t) denotes the EV position at the beginning of the MPC iteration and T is the sampling time, and $k = t+1, .., t+N$.

The velocity and heading references remain constant for all prediction steps:

$$v_{\text{ref},k} = V_{\text{ref}}, \qquad \phi_{\text{ref},k} = \phi_{\text{ref}} \tag{8}$$

Thus, the complete reference trajectory used by the MPC is

$$X_{\text{ref},k} = \begin{bmatrix} p_{x,\text{ref},k} \\ p_{y,\text{ref},k} \\ V_{\text{ref}} \\ \phi_{\text{ref}} \end{bmatrix}, \qquad k = t+1, \ldots, N \tag{9}$$

D. Model Predictive Control Formulation

The model predictive control (MPC) problem is formulated over a finite prediction horizon of length N. At each time step t, the controller computes an optimal sequence of control inputs $\{\mathbf{u}_0, \mathbf{u}_1, \ldots, \mathbf{u}_{N-1}\}$ by minimizing a finite-horizon cost function subject to vehicle dynamics, physical constraints, and safety constraints. For notational simplicity, all state and input variables indexed by k denote predicted quantities along the horizon at time t.

1) Objective Function: The MPC controller minimizes a finite-horizon objective that penalizes both tracking error and control effort. The total cost function is defined as

$$J = \sum_{k=0}^{N-1} \ell(\mathbf{x}_k, \mathbf{u}_k) + \ell_f(\mathbf{x}_N), \tag{10}$$

where the stage cost is given by

$$\ell(\mathbf{x}_k, \mathbf{u}_k) = \left(\mathbf{x}_k - \mathbf{x}_k^{\text{ref}}\right)^\top Q \left(\mathbf{x}_k - \mathbf{x}_k^{\text{ref}}\right) + \mathbf{u}_k^\top R\, \mathbf{u}_k, \tag{11}$$

and the terminal cost is defined as

$$\ell_f(\mathbf{x}_N) = \left(\mathbf{x}_N - \mathbf{x}_N^{\text{ref}}\right)^\top P_f \left(\mathbf{x}_N - \mathbf{x}_N^{\text{ref}}\right). \tag{12}$$

Here, $\mathbf{x}_k^{\text{ref}}$ denotes the reference state at prediction step k, $Q \succeq 0$ and $R \succ 0$ are the stage cost weighting matrices, and $P_f \succeq 0$ is the terminal cost matrix.

2) Dynamic Constraints: The evolution of the EV state along the prediction horizon is constrained by the linearized vehicle dynamics derived in Section II-B2. The predicted state must satisfy

$$\mathbf{x}_{k+1} = A_t\,\mathbf{x}_k + B_t\,\mathbf{u}_k + \delta_t, \quad k = 0, \ldots, N-1, \tag{13}$$

where A_t, B_t, and δ_t are obtained by linearizing the nonlinear vehicle model at the current state and input. The resulting matrices are held constant over the entire prediction horizon to preserve the linear equality constraint structure and maintain a quadratic programming (QP) formulation.

3) Physical and Map Constraints: The predicted state and input trajectories are required to remain within their admissible sets at all prediction steps:

$$\mathbf{x}_k \in \mathcal{X}, \quad \mathbf{u}_k \in \mathcal{U}, \quad k = 0, \ldots, N-1. \tag{14}$$

The sets $\mathcal{X}$ and $\mathcal{U}$ encode physical vehicle limitations as well as map-based constraints, ensuring that all predicted trajectories are feasible and safe with respect to vehicle capabilities and road geometry.

4) Safety Constraints: Collision avoidance between the EV and an OV is enforced through a geometric safety constraint requiring a minimum separation distance:

$$\left\| p_k^{\text{EV}} - p_k^{\text{OV}} \right\|_2 \geq d_{\min}, \quad k = 1, \ldots, N, \tag{15}$$

where p_k^{EV} and p_k^{OV} denote the positions of the EV and OV at prediction step k, and $d_{\min}$ is the minimum allowable distance.

Since the constraint in (15) is nonlinear and nonconvex, convex reformulations are employed to ensure real-time solvability within the MPC framework.

a) Gaussian Chance Constraint: When OV motion is uncertain and represented by a multi-modal predictor, the predictor provides a finite set of candidate future OV trajectories $\{p_k^{(s),\text{OV}}\}_{s=1}^{S}$, each associated with a probability weight w_s. These trajectories correspond to distinct latent behavioral hypotheses of the OV (e.g., yielding, proceeding, or stopping), rather than explicitly labeled motion modes.

For the purposes of the proposed B-MPC formulation, each trajectory is treated as a realization of a latent mode, and the resulting mixture distribution over OV positions at prediction step k is approximated by a Gaussian random variable

$$p_k^{\text{OV}} \sim \mathcal{N}\big(\mu_k^{\text{OV}},\, \Sigma_k^{\text{OV}}\big)\,. \tag{16}$$

with mean and covariance computed as

$$\begin{aligned} \mu_k^{\text{OV}} &= \sum_{s=1}^{S} w_s\, p_k^{(s),\text{OV}}, \\ \Sigma_k^{\text{OV}} &= \sum_{s=1}^{S} w_s \left(p_k^{(s),\text{OV}} - \mu_k^{\text{OV}}\right)\left(p_k^{(s),\text{OV}} - \mu_k^{\text{OV}}\right)^{\top}. \end{aligned} \tag{17}$$

The probabilistic safety requirement enforces that the EV maintains a distance of at least $d_{\min}$ from the OV with probability no smaller than $1-\varepsilon$:

$$\mathbb{P}\big(\hat{n}_k^{\top}\left(C\,\mathbf{x}_k - p_k^{\text{OV}}\right) \geq d_{\min}\big) \geq 1-\varepsilon, \tag{18}$$

where $\hat{n}_k$ denotes the unit normal vector of the linearized safety constraint

$$\hat{n}_k = \frac{C\,\mathbf{x}_k - p_k^{\text{OV}}}{||C\,\mathbf{x}_k - p_k^{\text{OV}}||_2} \tag{19}$$

and C extracts the EV position from the state vector.

Since the scalar random variable $\hat{n}_k^{\top}(C\,\mathbf{x}_k - p_k^{\text{OV}})$ is Gaussian, the chance constraint (18) admits the following deterministic reformulation:

$$\hat{n}_k^{\top} C\,\mathbf{x}_k \geq d_{\min} + \hat{n}_k^{\top}\mu_k^{\text{OV}} + \Phi^{-1}(1-\varepsilon)\sqrt{\hat{n}_k^{\top}\Sigma_k^{\text{OV}}\hat{n}_k}, \tag{20}$$

where $\Phi^{-1}(\cdot)$ denotes the inverse cumulative distribution function of the standard normal distribution.

5) MPC Optimization Problem Summary: Combining the objective function, dynamic constraints, feasibility constraints, and safety constraints, the MPC problem solved at each time step is formulated as

$$\begin{aligned} \min_{\{\mathbf{u}_k\}_{k=0}^{N-1}} \quad & \sum_{k=0}^{N-1} \ell(\mathbf{x}_k, \mathbf{u}_k) + \ell_f(\mathbf{x}_N) \\ \text{s.t.} \quad & \mathbf{x}_{k+1} = A_t\,\mathbf{x}_k + B_t\,\mathbf{u}_k + \delta_t, \\ & \mathbf{x}_k \in \mathcal{X}, \quad \mathbf{u}_k \in \mathcal{U}, \\ & \text{Safety via (20)} \end{aligned} \tag{21}$$

III. Single-Branch Bayesian Model Predictive Control with Multi-Modal Predictions

This session proposes a receding-horizon *Single-Branch Bayesian Model Predictive Control (BMPC)* framework that integrates multi-modal, probabilistic trajectory predictions of surrounding traffic participants into a standard MPC formulation. At each control step, a learned predictor provides a finite set of candidate future trajectories of the OV, each associated with a probability weight representing alternative hypotheses of the OV's motion over the prediction horizon.

Unlike scenario-tree or multi-policy stochastic MPC approaches [13], BMPC adopts a single-branch structure. Instead of optimizing separate control sequences for each predicted trajectory, the multi-modal predictions are aggregated into a single representative OV trajectory via probability-weighted averaging. The resulting mean trajectory and covariance are computed as in (17) and incorporated into the MPC problem through probabilistic safety constraints.

Algorithm 1 Single-Branch Bayesian Model Predictive Control (BMPC)

Require: Current EV state $\mathbf{x}_t$, previous control input $\mathbf{u}_{t-1}$, prediction horizon N, reference trajectory $\mathbf{x}^{\text{ref}}_{t+1:t+N}$, current OV states $\mathbf{x}^{\text{OV}}_t$, admissible state and input sets $\mathcal{X}, \mathcal{U}$

Ensure: Applied control input $\mathbf{u}_t$

1: **for** each time step $t = 0, 1, 2, \dots$ **do**

2: **OV prediction:**

$$\{\mathbf{x}^{(i)}_{t+1:t+N}, w_i\}_{i=1}^{M} \leftarrow \text{NN}(\mathbf{x}^{\text{OV}}_t), \quad \text{via (2)}$$

where M is the number of predicted trajectories per OV.

3: **Prediction aggregation:**

$$\mu^{\text{OV}}_k \leftarrow \mathbb{E}\left[\{\mathbf{x}^{(i)}_{t+1:t+N}, w_i\}_{i=1}^{M}\right]$$
$$\Sigma^{\text{OV}}_k \leftarrow \text{Cov}\left[\{\mathbf{x}^{(i)}_{t+1:t+N}, w_i\}_{i=1}^{M}\right], \quad \text{via (17)}$$

4: **Model linearization:**

$$A_t,\ B_t,\ C_t,\ \delta_t \leftarrow f_{\text{lin}}(\mathbf{x}_t, \mathbf{u}_{t-1}), \quad \text{via (13)}$$

5: **MPC optimization:**

$$\mathbf{u}^{\star}_k \leftarrow \text{QP}\left(\mathbf{x}_t, \mathbf{x}^{\text{ref}}_k, A_t, B_t, C, \delta_t, \mathcal{X}, \mathcal{U}, \mu^{\text{OV}}_k, \Sigma^{\text{OV}}_k\right) \quad \text{via (21)}$$

6: **Control application:**

$$\mathbf{u}_t \leftarrow \mathbf{u}^{\star}_{t|k}$$

7: **System update:**

$$\mathbf{x}_{t+1} \leftarrow f\left(\mathbf{x}_t, \mathbf{u}_t\right), \quad \text{via (3)}$$

8: **end for**

The MPC then computes a single control sequence by minimizing the finite-horizon objective (10), subject to linearized vehicle dynamics (4), physical and map constraints, and the deterministic reformulation of the Gaussian chance constraint (20). This formulation enforces a minimum safety distance from the OV with a specified confidence level, explicitly accounting for OV uncertainty while preserving convexity and real-time solvability.

Algorithm 1 summarizes the control loop: at each time step, the controller (i) receives updated multi-modal OV predictions, (ii) computes the probability-weighted mean and covariance, (iii) solves a single MPC optimization over the horizon, and (iv) applies only the first control input before repeating at the next step.

Although multiple OV hypotheses are considered internally, the scenario tree is collapsed at each step, resulting in a single optimization and applied control input. Compared to nominal MPC [14], which assumes a deterministic environment, BMPC explicitly incorporates OV uncertainty via probabilistic predictions and chance-constrained safety, enabling adaptive and safe EV behavior while maintaining a computational complexity comparable to standard linear MPC.

A. Design of C-MPC

The C-MPC adds contingency plans for when the environment isn't behaving as predicted. In this case it will be when an OV goes in the EVs way when the MPC has predicted that it wouldn't and the measures to avoid collision is beyond the limits of the MPC. This means that for every instance the C-MPC plans at least one contingency plan, see Fig. 6.

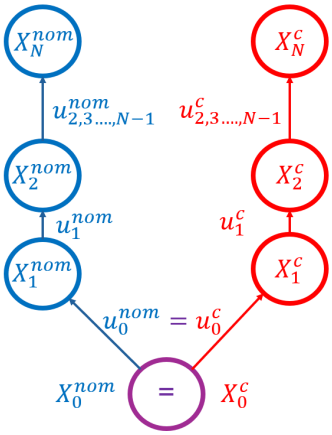

Fig. 6: The nominal (nom) and the contingency (c) plan by the C-MPC generated at each instance

At each MPC iteration, the C-MPC computes both a nominal trajectory and at least one contingency trajectory in parallel for the EV. The nominal plan is executed as long as the predicted behavior of the OV's remains consistent, no sudden changes in behavior, with the assumptions made during optimization. If a deviation is detected that renders the nominal plan infeasible or unsafe, the controller immediately switches to the corresponding contingency plan. This mechanism ensures that a feasible and safety-oriented control action is always available, even under unexpected OV behavior.

1) Contingency Plan: In the contingency logic, $OV = \text{Go}$ denotes the situation in which the obstacle vehicle is predicted to enter the conflict zone, contrary to the behavior predicted by the MPC at the previous control iteration. This discrepancy between predicted and observed obstacle behavior is interpreted as a prediction failure and triggers the evaluation of contingency conditions, , which are formalized in Algorithm 2.

The first contingency mode, *Stopping*, is activated when the required deceleration exceeds a predefined fraction c_{stop} of the maximum achievable deceleration. The parameter c_{stop} is a tunable threshold that determines how aggressively the ego vehicle is allowed to brake before abandoning the nominal MPC behavior. When this mode is selected, the ego vehicle applies maximum feasible deceleration in order to stop prior to entering the conflict zone.

The second contingency mode, *Avoiding*, is selected when the required deceleration exceeds the physical

Algorithm 2 Contingency Mode Selection Logic

Require: Obstacle vehicle state OV, required deceleration a_{req}, maximum deceleration $a_{\max}$, stopping threshold c_{stop}
Ensure: Operating mode $Mode$
1: **if** $OV = \text{Go}$ **and** $\frac{a_{\text{req}}}{a_{\max}} > c_{\text{stop}}$ **then**
2: $\quad Mode \leftarrow$ Stopping
3: **end if**
4: **if** $OV = \text{Go}$ **and** $\frac{a_{\text{req}}}{a_{\max}} > 1$ **then**
5: $\quad Mode \leftarrow$ Avoiding
6: **end if**

deceleration limit, indicating that a full stop cannot be achieved before a potential collision with the obstacle vehicle. In this case, the ego vehicle executes an evasive acceleration maneuver to clear the conflict area and does not reduce its speed until the obstacle vehicle has been safely passed.

These two contingency plans has been verified in the table(7). The contingency first checks if it can stop before the intersection and if that fails it checks if it can stop before entering the safety distance to the OV and if it can't do that, only then does it select to enter the avoid plan. This results in that only when the breaking distance is longer than the width of the unoccupied road lane ($1[m]$) does it need to be able to manage a difference in reached distance longer than the width of the OV ($3[m]$). In table (7) it can be seen that our critical speed is between $16[km/h]$ and $25[km/h]$ when the EV can't stop in less than $1[m]$ or clear $3[m]$. But calculating the time i would take a stationary OV to hit the EV at the worst possible time is equal to $1.0560[s]$ and in that time with the EV initially traveling at $16[km/h]$, the EV would be able to clear $5.69[m]$ which is longer than the width of the lane of the OV ensuring that these two contingency plans are sufficient for this scenario.

EV velocity in [km/h]	Stoping distance [m]	Reached distance when accelerating in the same time frame [m]	Difference in distance [m]
5	0,10	0,32	0,23
10	0,38	1,03	0,65
15	0,86	2,12	1,26
16	0,98	2,39	1,41
20	1,53	3,60	2,07
25	2,39	5,46	3,07
45	7,74	16,74	9,00
70	18,73	39,50	20,77

Fig. 7: The contingency plan verification

IV. Simulation Environments

This section presents the two simulation environments used to evaluate the proposed controllers. The general simulation setup and test scenarios are kept identical across both environments to enable a consistent comparison.

A. Environment 1: Synthetic Intersection Scenario

The first simulation environment consists of a simplified cross-section intersection, illustrated in Fig. 8, involving one EV and one OV. Both vehicles are initialized at equal distances from the center of the intersection, such that a collision is guaranteed if no control action is applied. This setup provides a controlled baseline for evaluating collision avoidance performance.

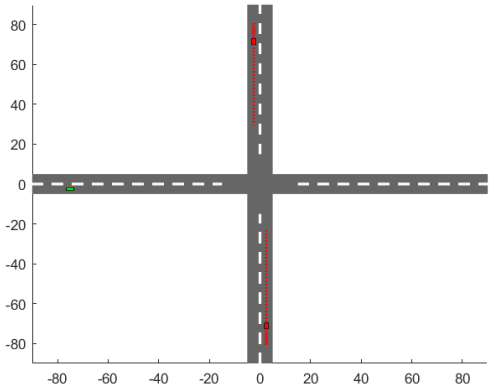

Fig. 8: Intersection during simulation.

The intersection scenario is evaluated under three distinct configurations, summarized in Table I, which vary the vehicle speeds and the stopping behavior of the OV.

TABLE I: Test scenarios for evaluating Single-Branch BMPC in the synthetic cross-section environment shown in Fig. 8.

Scenario	Speed [kph]	OV stopping
1	45	True
2	45	False
3	70	False

Multi-modal trajectory predictions for the OV are generated using the Trajectron-based predictor described in Section II-A2, providing probabilistic hypotheses of the OV's future motion.

B. Environment 2: Real-World Intersection Scenario

The second simulation environment is derived from real-world traffic data provided by the InD dataset [15]. The layout of this environment is shown in Fig. 9. Compared to the synthetic scenario, this environment introduces increased realism in terms of road geometry and traffic interactions.

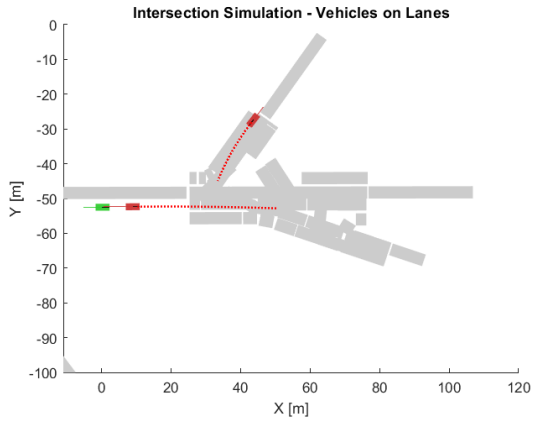

Fig. 9: Map of the InD-based simulation environment.

To reduce computational complexity, the Trajectron-based predictor is not used in this environment. Instead, the binary prediction model described in Section II-A1 is employed to capture the OV's high-level motion intent.

In this scenario, the OV directly ahead of the EV proceeds straight at a constant velocity, while the second OV approaching from above comes to a complete stop before merging into the lane occupied by the EV. This environment evaluates the controller's performance under more realistic traffic dynamics but reduced prediction richness.

The OV is manually intilized to simulate a case where a collision is probable.

V. Results

This section presents the results of the proposed framework in two stages. The first part evaluates the training performance of the neural network–based prediction models, focusing on prediction accuracy. The second part examines the closed-loop performance of the proposed controllers when combined with these predictors, using the simulation scenarios introduced in Section IV.

A. B-MPC

The Single-Branch B-MPC is evaluated in two simulation environments, using consistent controller tuning and safety constraint settings across both cases. The prediction horizon is set to $N = 30$, and the Gaussian chance constraint (20) is relaxed with an allowable violation probability of $\varepsilon = 0.1$ to account for linearization errors in the safety constraint and to increase feasibility. The minimum allowable distance between the EV and OV is set to $d_{\min} = 5\,\text{m}$.

The controller is tuned according to the finite-horizon objective in (10), with the weighting matrices selected as:

$$\begin{aligned} Q &= 1 \times 10^{-5}\,\text{diag}([1,\,1,\,1,\,1 \times 10^{10}]) \\ R &= 10\,\text{diag}([1,\,10]) \\ P_f &= 100\,\text{diag}([1,\,1,\,1,\,1]) \end{aligned} \tag{22}$$

1) Simulation Environment 1: The results from the tests discussed in Section IV-A are presented below.

a) Scenario 1: The first scenario in Table I is illustrated in Fig. 10, in which the OV comes to a complete stop at the intersection and waits for the EV to pass.

Notably, the EV exhibits a strong deceleration, around time instance $t = 35$, even though the OV is stationary. This behavior can be attributed to the long prediction horizon employed by the B-MPC controller: the neural predictor generates multi-modal trajectories in which the OV does not stop immediately, and the controller accounts for these probabilistic predictions when enforcing safety constraints. Consequently, the EV adopts a conservative deceleration to proactively mitigate potential collisions, demonstrating how the controller integrates uncertainty in other-vehicle behavior into its decision-making.

b) Scenario 2: With the same velocity, the scenario is changed to allow the OV continue through the intersection. The resulting states and control inputs are shown in Fig. 11. Here, once again, the EV acts by a negative acceleration. However, the deceleration is significantly less pronounced compared to Fig. 10. Instead, the EV is able to maintain a higher speed while still satisfying the safety constraint, as evidenced by the distance profile in the upper plot of Fig. 11. This behavior reflects a less conservative response, allowing the EV to minimize delay while maintaining safety under continued OV motion.

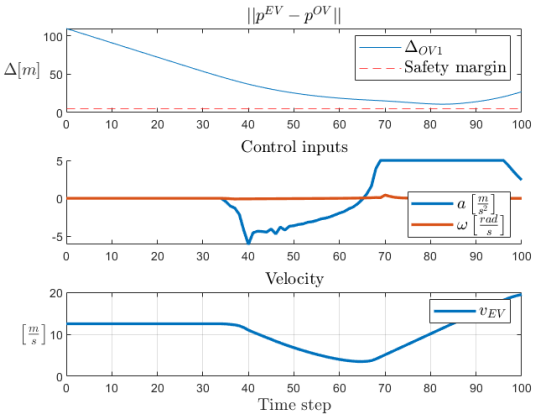

Fig. 10: Distance to OV, control inputs and velocity of EV. EV initially traveling at 45 km/h and OV stopping.

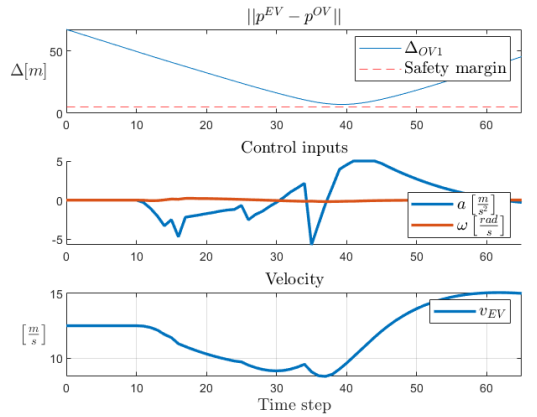

Fig. 11: Distance to OV, control inputs and velocity of EV. Both cars initially traveling at 45 km/h and OV driving through intersection without stopping.

c) Scenario 3: In the final scenario, both the EV and OV travel at an initial speed of 70 km/h. The resulting behavior is illustrated in Fig. 12. The EV exhibits dynamics similar to those observed in Scenario 2, characterized by a modest reduction in velocity and minimal steering intervention to ensure safe passage through the intersection.

After the EV has safely passed the OV, the controller commands a higher acceleration to recover the desired velocity and converge back to the reference trajectory. This demonstrates the controller's ability to balance safety and performance, applying conservative actions only when necessary and aggressively returning to nominal behavior once constraints are no longer active.

2) Simulation Environment 2: The performance of the Single-Branch BMPC in the real-world intersection scenario described in Section IV-B is illustrated in Fig. 13. The EV initially travels at 32.4 km/h, while the OV approaching from the adjacent lane decelerates and comes to a complete stop prior to merging.

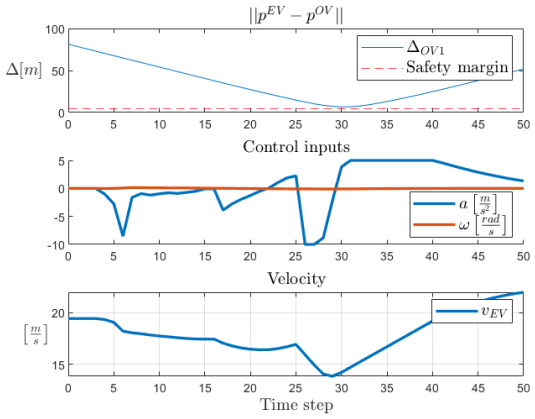

Fig. 12: Distance to OV, control inputs and velocity of EV. Both cars traveling at 70 km/h and OV driving through intersection without stopping.

As shown in the distance profile (top figure) the B-MPC handles the situation wihtout a collision. However, it is noteworthy that instead of reducing its velocity to allow the OV to merge, the EV accelerates slightly to move ahead of the merging vehicle.

Although the behavior of the EV deviates from the results seen in Section V-A1, the results from real-world intersection scenario still demonstrate that the proposed B-MPC framework generalizes beyond synthetic scenarios and remains effective in a realistic traffic environment, even when relying on reduced prediction fidelity. The controller maintains safety while avoiding overly aggressive braking, highlighting its robustness to uncertainty in OV behavior.

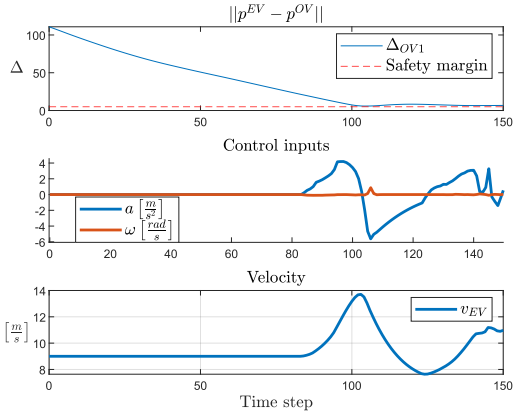

Fig. 13: Simulation of B-MPC using the InD data-set. Distance to OV, control inputs and velocity of EV. EV initially traveling at 32.4 km/h and OV stopping.

B. C-MPC

In this section the outcome of two different sets of simulations will be presented. One that focuses on the comparison between the B-MPC and C-MPC. The other simulation focuses on a comparison between the C-MPC formulation with and without the contingency activated.

1) Simulation Environment 1: To assess the behavior of the C-MPC controller, time-domain simulation results are presented for three representative scenarios in the synthetic intersection environment described in Section IV-A. The evaluated cases correspond to the configurations summarized in Table I.

For each scenario, the applied control inputs and the resulting state trajectories of the EV are examined. The results illustrate how the C-MPC controller adapts its control actions in response to varying traffic dynamics and vehicle speeds, while consistently maintaining stable and feasible behavior. In line with the B-MPC simulations, a prediction horizon of N=30 is used throughout all experiments to ensure a consistent basis for comparison.

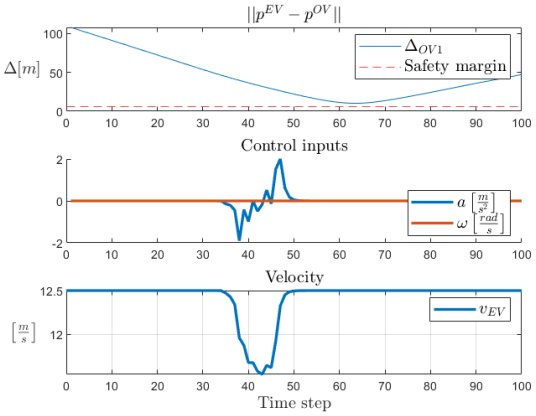

Fig. 14: Distance to OV, control inputs and velocity of EV. EV initially traveling at 45 km/h and OV stopping.

As shown in Fig. 14, the sudden change in the OV's trajectory, caused by its abrupt stop, triggers a distinct control response from the C-MPC controller. The controller applies a strong deceleration to the EV in order to maintain a safe longitudinal distance while respecting system constraints. Despite the aggressive maneuver, the state trajectories remain stable and feasible, demonstrating the controller's ability to handle sudden and unforeseen disturbances.

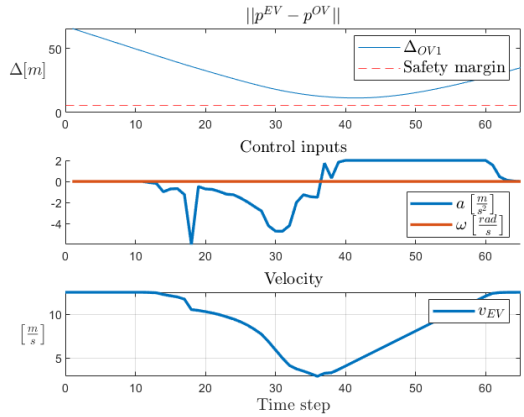

Fig. 15: Distance to OV, control inputs and velocity of EV. Both cars initially traveling at 45 km/h and OV driving through intersection without stopping.

For the second case seen in Fig. 15 at 45 kph , the controller exhibits moderate control actions, with the vehicle velocity closely tracking the reference and minimal variation in yaw-rate. However, it has a slight overreaction in the start of the control sequence. It does also show proof of a safe distance between the EV and OV.

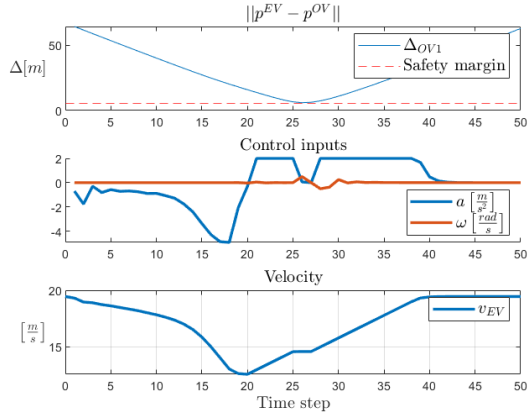

Fig. 16: Distance to OV, control inputs and velocity of EV. Both cars traveling at 70 km/h and OV driving through intersection without stopping.

The third scenario evaluates controller performance at a higher speed of 70 km/h, as shown in Figure 16. Compared to the lower-speed cases, the controller generates more pronounced control actions to account for the increased vehicle dynamics and reduced reaction time. Nevertheless, the EV remains stable, the reference velocity is accurately tracked, and safety constraints are satisfied throughout the maneuver. This demonstrates the scalability and robustness of the C-MPC controller across different operating speeds.

2) Simulation Environment 2: Figure 17 illustrates a representative simulation of the C-MPC controller operating in the real-world intersection environment derived from the InD dataset. The EV is initialized at a velocity of 32.4 km/h, while surrounding traffic participants follow trajectories obtained directly from the recorded dataset.

Unlike the synthetic intersection scenarios, this environment does not rely on parametrized or scripted obstacle behavior. Instead, the controller receives sensor-equivalent measurements reconstructed from real-world vehicle trajectories, introducing realistic traffic dynamics and interaction patterns. The purpose of this simulation is therefore not to perform a detailed performance comparison, but to demonstrate that the C-MPC controller can operate stably and generate feasible control inputs when driven by real-world data.

As shown in Figure 17, the controller maintains a safe distance to the obstacle vehicle while producing smooth and physically feasible control actions. No infeasible solutions or instability were observed during the simulation, indicating that the proposed control framework is compatible with real-world, sensor-based traffic inputs.

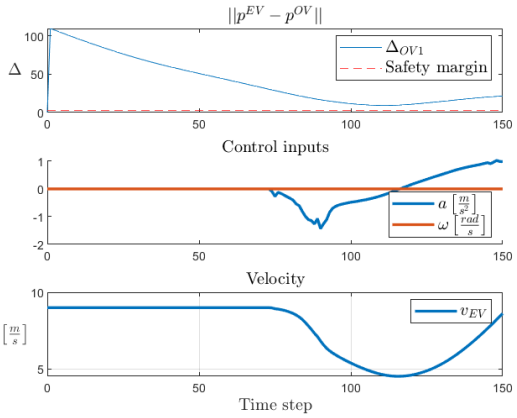

Fig. 17: Simulation of C-MPC using the InD data-set. Distance to OV, control inputs and velocity of EV. EV initially traveling at 32.4 km/h.

3) Verification of the Contingency Plan: This section verifies the correct operation of the proposed contingency plan. The objective is to demonstrate that the controller transitions to the appropriate operating mode in critical situations and successfully prevents collisions under the considered scenarios. This comparison focuses on the C-MPC with and without the contingency plan to evaluate both the nominal MPC and the contingency mechanism.

A total of 100 simulation runs using the scenarios in Section IV-A were conducted for each controller configuration, consisting of 50 runs per scenario. Scenario 1 corresponds to obstacle vehicles crossing with constant velocity, while Scenario 2 includes an obstacle vehicle that accelerates unexpectedly.

In Scenario 1 (constant-velocity crossing), both configurations successfully cleared the intersection in all

runs $(50/50)$.

In Scenario 2 (unexpected acceleration), the nominal MPC without contingency failed in $12/50$ runs, whereas enabling the contingency plan reduced the number of failures to $2/50$ runs, this can be seen in table II.

These results confirm that the contingency plan operates as intended. It does not affect nominal behavior in scenarios with constant obstacle velocity, while substantially reducing the collision rate when unexpected obstacle behavior is introduced. In figure 18, distance, velocities and control inputs can be seen to show how the vehicle is affected by the contingency plan.

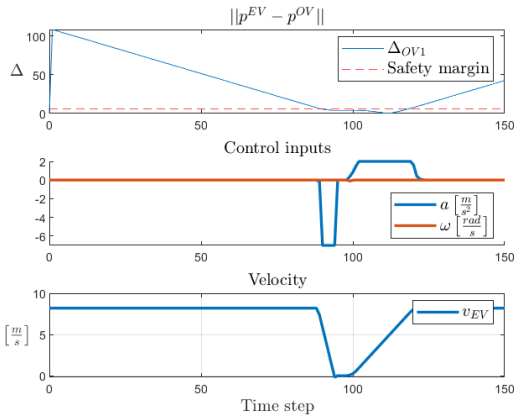

Fig. 18: KPI's of CMPC using the contingency plan.

C. Comparison Between B-MPC and C-MPC

A general comparison between the B-MPC and C-MPC controllers reveals similar closed-loop behavior across the evaluated scenarios. In both cases, the controllers are able to maintain stability, satisfy system constraints, and ensure safe interaction between the EV and the OV. Reference tracking performance is comparable, and neither approach exhibits infeasible or unstable behavior in the tested operating conditions. There are however some minor differences in behavior.

1) Scenario 1: Comparing the results from the first scenario from the B-MPC (10) and from the C-MPC (15) the results are very similar. Both keeps a safe distance but the control inputs from the C-MPC are smaller keeping within an acceleration of -2 and 2 $[m/s^2]$ while the B-MPC operates over a span of -5 and 5 $[m/s^2]$. The C-MPC also clears the intersection faster than the B-MPC.

TABLE II: Simulation outcomes over 50 runs per scenario. A success indicates that the EV cleared the interaction without collision; a failure indicates a collision.

Method	S1 (CV)	S2 (UA)
MPC (no cont.)	50/50 (100%)	38/50 (76%)
CMPC (cont.)	50/50 (100%)	48/50 (96%)

2) Scenario 2: The results from the second scenario from the B-MPC (11) and from the C-MPC (14) both maintains safe distances but the C-MPC doesn't accelerates as fast as the B-MPC only having a max acceleration of 2 $[m/s^2]$ compared to the B-MPC:s 5 $[m/s^2]$.

3) Scenario 3: Both the B-MPC (12) and the C-MPC (16) manage to maintain a safe distance to the OV and have a similar velocity profile where they de-accelerates long before the intersection and then has a harsher braking before starting to accelerates again. The main difference is the B-MPC having a harsher acceleration being between -10 and 5 $[m/s^2]$ while the C-MPC stays between -5 and 2 $[m/s^2]$.

4) Simulation Environment 2: In the real-world intersection scenario derived from the InD dataset, both controllers enable the EV to safely negotiate the intersection without violating the safety constraints. Despite achieving the same safety objective, the controllers exhibit qualitatively different behaviors.

The C-MPC adopts a conservative strategy by applying a mild deceleration, allowing the EV to yield and pass behind the OV. In contrast, the Single-Branch B-MPC executes a proactive maneuver by accelerating slightly to pass ahead of the OV. These differing strategies reflect the underlying control philosophies of the two approaches.

In terms of control effort, both controllers generate moderate actuation. However, the B-MPC exhibits marginally higher control magnitudes, which can be attributed to its less restrictive acceleration bounds, allowing inputs in the range $-10 \leq a \leq 5\,[m/s^2]$, compared to $-5 \leq a \leq 2\,[m/s^2]$ for the C-MPC. This increased control authority enables more aggressive maneuvers while still maintaining safety.

VI. Discussion

Both the the B-MPC and the C-MPC manage to keep a safe distance to the OV but does so slightly differently. The B-MPC generally have harsher control inputs compared to the C-MPC with both a higher acceleration and de-acceleration. This doesn't have to be an attribute to the B-MPC logic as there are parameters to be tuned and getting both controllers to work with the exact same parameters wasn't doable in this report. But it does stand to reason that the C-MPC can focus more on smaller control inputs as it has it's contingency plan to fall back on if something unexpected happens while the B-MPC must be able to solve every situation in the moment and therefore must be a bit more cautious.

A. Computational Considerations

A part that can have a big impact on performance that hasn't been evaluated in this report is the computing power needed for the different controllers. As one controller might be slightly better but demands twice the computing power to achieve it resulting in it actually being the worse of the two controllers.

B. Role and Evaluation of the Contingency Mechanism

One noteworthy aspect of the C-MPC controller is that the contingency mechanism is rarely activated. This observation can be interpreted from two perspectives. On the one hand, the seldom activation indicates that the nominal MPC controller is capable of handling the vast majority of encountered scenarios on its own. From a safety standpoint, this is a highly desirable property: if the contingency plan is not required, it implies that potentially critical situations are successfully managed before reaching emergency conditions.

On the other hand, the rare activation of the contingency plan complicates a thorough evaluation of its performance. The contingency strategy is only triggered in particularly challenging or extreme scenarios, which occur infrequently in the tested cases. As a result, there are limited opportunities to systematically assess its effectiveness, robustness, and potential conservatism under a wide range of operating conditions.

Overall, while the limited use of the contingency mechanism reflects positively on the nominal controller's capability and safety performance, it also highlights the need for carefully designed stress-test scenarios to fully characterize and validate the contingency behavior.

C. Acceleration of B-MPC

In Simulation Environment 2 (Fig. 13), the EV exhibits a slight acceleration to move ahead of the merging OV, rather than decelerating when controlled by the B-MPC. This behavior is believed to arise from the timing of the predicted OV trajectories: the controller receives updates indicating the merge too late to safely reduce speed. Consequently, the BMPC performs a proactive maneuver to maintain safety. This example highlights the controller's capability to handle prediction delays and limited prediction fidelity in real-world traffic, demonstrating robust and anticipatory behavior under uncertainty.

1) Tuning of B-MPC Motivation: The behavior of the BMPC is largely determined by the tuning of its finite-horizon cost function (22). To allow the EV sufficient flexibility in avoiding the OV, the state cost matrix Q imposes minimal penalties on deviations in position and velocity, while maintaining a high penalty on the heading angle to discourage overly aggressive steering and ensure stability.

The control cost matrix R is chosen to balance smooth actuation with the ability to execute rapid evasive maneuvers when necessary. The heading angle ϕ is heavily penalized, rather than the heading rate ω, to ensure that when the EV must steer to avoid the OV, it can quickly return to the center of the lane instead of drifting toward the lane boundary. This is facilitated by the relatively low penalty on steering rate, which allows for fast adjustments without overly aggressive control inputs. Similarly, the terminal cost P_f emphasizes convergence to the reference trajectory at the end of the prediction horizon, applying a strong penalty to guide the EV back to the desired path and maintain overall trajectory fidelity.

In addition, the Gaussian chance constraint relaxation parameter ε is set relatively high to compensate for linearization errors in the geometric safety constraint (15), which is inherently nonlinear and nonconvex. Linearizing this constraint introduces modeling errors and some conservativeness; the increased ε mitigates these effects, ensuring both feasibility and safety. Collectively, these tuning choices achieve a careful trade-off between trajectory adherence, safety under uncertainty, and controller feasibility.

VII. Conclusion

In conclusion both the B-MPC and the C-MPC achieved the goal of maintaining a safe distance but the C-MPC has been shown to result in smaller control inputs and would therefore give a smoother car ride. But this has been a very limited research and more scenarios needs to be tested with the computational demand in consideration to enable a solid foundation to determine a better controller and their strengths and weaknesses.

References

[1] A. Alahi, K. Goel, V. Ramanathan, A. Robicquet, L. Fei-Fei, and S. Savarese, "Social lstm: Human trajectory prediction in crowded spaces," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[2] A. Gupta, J. Johnson, L. Fei-Fei, S. Savarese, and A. Alahi, "Social gan: Socially acceptable trajectories with generative adversarial networks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.

[3] K. Mangalam, Y. An, H. Girase, and J. Malik, "Pecnet: Predicting multiple future trajectories using endpoint conditioning," in *European Conference on Computer Vision (ECCV)*, 2020.

[4] T. Phan-Minh, S. R. Bulo, L. Porzi, and P. Kontschieder, "Covernet: Multimodal behavior prediction using trajectory sets," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.

[5] H. Cui, V. Radosavljevic, F. Y. Chou, T.-Y. Lin, T. Nguyen, T.-H. Huang, and N. Djuric, "Multipath: Multiple probabilistic anchor trajectory hypotheses for behavior prediction," in *Conference on Robot Learning (CoRL)*, 2019.

[6] H. Cui, T.-Y. Lin, V. Radosavljevic, F. Y. Chou, and N. Djuric, "Multimodal trajectory predictions for autonomous driving using multipath++," in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2021.

[7] J. Gao, C. Sun, H. Li, and M. Tomizuka, "Vectornet: Encoding hd maps and agent dynamics from vectorized representation," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.

[8] M. Liang, B. Jiang, Y. Chen, F. Wang, and R. Urtasun, "Lanegcn: A context-aware trajectory prediction network," in *European Conference on Computer Vision (ECCV)*, 2020.

[9] J. Gu, C. Sun, H. Zhao, and M. Tomizuka, "Densetnt: End-to-end trajectory prediction from dense goal sets," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2021.

[10] T. Salzmann, B. Ivanovic, P. Chakravarty, and M. Pavone, "Trajectron++: Dynamically-feasible trajectory forecasting with heterogeneous data," in *European Conference on Computer Vision (ECCV)*, 2020.

[11] H. Bekker. (2025) best selling car europe. [Online]. Available: https://www.best-selling-cars.com/europe/2024-full-year-europe-top-50-best-selling-car-models/

[12] fastestlaps. (2025) fastestlaps. [Online]. Available: https://fastestlaps.com/models/dacia-sandero-tce-90

[13] Y. Chen, U. Rosolia, W. Ubellacker, N. Csomay-Shanklin, and A. D. Ames, "Interactive multi-modal motion planning with branch model predictive control," *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 5365–5372, 2022.

[14] MathWorks. (2025) What is model predictive control? Accessed: 2025-11-04. [Online]. Available: https://se.mathworks.com/help/mpc/gs/what-is-mpc.html

[15] J. Bock, R. Krajewski, T. Moers, S. Runde, L. Vater, and L. Eckstein, "The ind dataset: A drone dataset of naturalistic road user trajectories at german intersections," in *2020 IEEE Intelligent Vehicles Symposium (IV)*, 2020, pp. 1929–1934.